sklearn.linear_model.LassoLars

class `sklearn.linear_model.`**`LassoLars`**(*alpha=1.0, *, fit_intercept=True, verbose=False, normalize='deprecated', precompute='auto', max_iter=500, eps=2.220446049250313e-16, copy_X=True, fit_path=True, positive=False, jitter=None, random_state=None*) [source]

Lasso model fit with Least Angle Regression a.k.a. Lars.

It is a Linear Model trained with an L1 prior as regularizer.

The optimization objective for Lasso is:

```
(1 / (2 * n_samples)) * ||y - Xw||^2_2 + alpha * ||w||_1
```

Read more in the User Guide.

Parameters:

alpha : *float, default=1.0*
Constant that multiplies the penalty term. Defaults to 1.0. `alpha = 0` is equivalent to an ordinary least square, solved by `LinearRegression`. For numerical reasons, using `alpha = 0` with the LassoLars object is not advised and you should prefer the LinearRegression object.

fit_intercept : *bool, default=True*
Whether to calculate the intercept for this model. If set to false, no intercept will be used in calculations (i.e. data is expected to be centered).

verbose : *bool or int, default=False*
Sets the verbosity amount.

normalize : *bool, default=True*
This parameter is ignored when `fit_intercept` is set to False. If True, the regressors X will be normalized before regression by subtracting the mean and dividing by the l2-norm. If you wish to standardize, please use `StandardScaler` before calling `fit` on an estimator with `normalize=False`.

> *Deprecated since version 1.0:* `normalize` was deprecated in version 1.0. It will default to False in 1.2 and be removed in 1.4.

precompute : *bool, 'auto' or array-like, default='auto'*
Whether to use a precomputed Gram matrix to speed up calculations. If set to `'auto'` let us decide. The Gram matrix can also be passed as argument.

max_iter : *int, default=500*
Maximum number of iterations to perform.

eps : *float, default=np.finfo(float).eps*
The machine-precision regularization in the computation of the Cholesky diagonal factors. Increase this for very ill-conditioned systems. Unlike the `tol` parameter in some iterative optimization-based algorithms, this parameter does not control the tolerance of the optimization.

copy_X : *bool, default=True*
If True, X will be copied; else, it may be overwritten.

fit_path : *bool, default=True*
If `True` the full path is stored in the `coef_path_` attribute. If you compute the solution for a large problem or many targets, setting `fit_path` to `False` will lead to a speedup, especially with a small alpha.

positive : *bool, default=False*
Restrict coefficients to be >= 0. Be aware that you might want to remove fit_intercept which is set True by default. Under the positive restriction the model coefficients will not converge to the ordinary-least-squares solution for small values of alpha. Only coefficients up to the smallest alpha value (`alphas_[alphas_ > 0.].min()` when fit_path=True) reached by the stepwise Lars-Lasso algorithm are typically in congruence with the solution of the coordinate descent Lasso estimator.

jitter : *float, default=None*
Upper bound on a uniform noise parameter to be added to the `y` values, to satisfy the model's assumption of one-at-a-time computations. Might help with stability.

rsion 0.23.

random_state : *int, RandomState instance or None, default=None*

Determines random number generation for jittering. Pass an int for reproducible output across multiple function calls. See Glossary. Ignored if `jitter` is None.

New in version 0.23.

Attributes:

alphas_ : *array-like of shape (n_alphas + 1,) or list of such arrays*

Maximum of covariances (in absolute value) at each iteration. `n_alphas` is either `max_iter`, `n_features` or the number of nodes in the path with `alpha >= alpha_min`, whichever is smaller. If this is a list of array-like, the length of the outer list is `n_targets`.

active_ : *list of length n_alphas or list of such lists*

Indices of active variables at the end of the path. If this is a list of list, the length of the outer list is `n_targets`.

coef_path_ : *array-like of shape (n_features, n_alphas + 1) or list of such arrays*

If a list is passed it's expected to be one of n_targets such arrays. The varying values of the coefficients along the path. It is not present if the `fit_path` parameter is `False`. If this is a list of array-like, the length of the outer list is `n_targets`.

coef_ : *array-like of shape (n_features,) or (n_targets, n_features)*

Parameter vector (w in the formulation formula).

intercept_ : *float or array-like of shape (n_targets,)*

Independent term in decision function.

n_iter_ : *array-like or int*

The number of iterations taken by lars_path to find the grid of alphas for each target.

n_features_in_ : *int*

Number of features seen during fit.

New in version 0.24.

feature_names_in_ : *ndarray of shape (`n_features_in_`,)*

Names of features seen during fit. Defined only when `X` has feature names that are all strings.

New in version 1.0.

See also:

`lars_path`

Compute Least Angle Regression or Lasso path using LARS algorithm.

`lasso_path`

Compute Lasso path with coordinate descent.

`Lasso`

Linear Model trained with L1 prior as regularizer (aka the Lasso).

`LassoCV`

Lasso linear model with iterative fitting along a regularization path.

`LassoLarsCV`

Cross-validated Lasso, using the LARS algorithm.

`LassoLarsIC`

Lasso model fit with Lars using BIC or AIC for model selection.

`sklearn.decomposition.sparse_encode`

Sparse coding.

Examples

```
>>> from sklearn import linear_model
>>> reg = linear_model.LassoLars(alpha=0.01, normalize=False)
>>> reg.fit([[-1, 1], [0, 0], [1, 1]], [-1, 0, -1])
LassoLars(alpha=0.01, normalize=False)
>>> print(reg.coef_)
[ 0.         -0.955...]
```

Toggle Menu

`fit`(X, y[, Xy])	Fit the model using X, y as training data.
`get_params`([deep])	Get parameters for this estimator.
`predict`(X)	Predict using the linear model.
`score`(X, y[, sample_weight])	Return the coefficient of determination of the prediction.
`set_params`(**params)	Set the parameters of this estimator.

`fit`(*X, y, Xy=None*) [source]

Fit the model using X, y as training data.

Parameters:

X : *array-like of shape (n_samples, n_features)*
Training data.

y : *array-like of shape (n_samples,) or (n_samples, n_targets)*
Target values.

Xy : *array-like of shape (n_samples,) or (n_samples, n_targets), default=None*
Xy = np.dot(X.T, y) that can be precomputed. It is useful only when the Gram matrix is precomputed.

Returns:

self : *object*
Returns an instance of self.

`get_params`(*deep=True*) [source]

Get parameters for this estimator.

Parameters:

deep : *bool, default=True*
If True, will return the parameters for this estimator and contained subobjects that are estimators.

Returns:

params : *dict*
Parameter names mapped to their values.

`predict`(*X*) [source]

Predict using the linear model.

Parameters:

X : *array-like or sparse matrix, shape (n_samples, n_features)*
Samples.

Returns:

C : *array, shape (n_samples,)*
Returns predicted values.

`score`(*X, y, sample_weight=None*) [source]

Return the coefficient of determination of the prediction.

The coefficient of determination R^2 is defined as $(1 - \frac{u}{v})$, where u is the residual sum of squares `((y_true - y_pred)** 2).sum()` and v is the total sum of squares `((y_true - y_true.mean()) ** 2).sum()`. The best possible score is 1.0 and it can be negative (because the model can be arbitrarily worse). A constant model that always predicts the expected value of `y`, disregarding the input features, would get a R^2 score of 0.0.

Parameters:

Toggle Menu

like of shape (n_samples, n_features)

Test samples. For some estimators this may be a precomputed kernel matrix or a list of generic objects instead with shape `(n_samples, n_samples_fitted)`, where `n_samples_fitted` is the number of samples used in the fitting for the estimator.

y : ***array-like of shape (n_samples,) or (n_samples, n_outputs)***
True values for `X`.

sample_weight : ***array-like of shape (n_samples,), default=None***
Sample weights.

Returns:

score : ***float***
R^2 of `self.predict(X)` wrt. `y`.

Notes

The R^2 score used when calling `score` on a regressor uses `multioutput='uniform_average'` from version 0.23 to keep consistent with default value of `r2_score`. This influences the `score` method of all the multioutput regressors (except for `MultiOutputRegressor`).

`set_params`(***params*) [source]

Set the parameters of this estimator.

The method works on simple estimators as well as on nested objects (such as `Pipeline`). The latter have parameters of the form `<component>__<parameter>` so that it's possible to update each component of a nested object.

Parameters:

****params :** ***dict***
Estimator parameters.

Returns:

self : ***estimator instance***
Estimator instance.

© 2007 - 2022, scikit-learn developers (BSD License). Show this page source